

Основные понятия

- **Криптология** - наука, которая делится на две науки: криптография и криптоанализ.
- **Криптография** - наука, которая изучает преобразования, делающие смысл информации непригодным для злоумышленника.
- **Криптоанализ** - наука, которая изучает нахождение смысла информации без доступа к секретному параметру системы (ключу).

- **Ключ** - уникальное состояние некоторого параметра системы, которое позволяет из множества возможных отображений выбрать нужное. Простыми словами, **ключ** - это секретный параметр системы шифрования.
- **Шифр** - мат. функция, которая преобразовывает открытый блок данных на основе секретного параметра (ключа). Другими словами, **шифр** - алгоритм, с помощью которого выполняется и шифрование и расшифрование данных.

- **Шифрование** — процесс нормального применения криптографического преобразования открытого текста на основе алгоритма и ключа, в результате которого возникает шифрованный текст.
- **Расшифровывание** — процесс нормального применения криптографического преобразования шифрованного текста в открытый.

- **Дешифрование (дешифровка)** — процесс извлечения открытого текста без знания криптографического ключа на основе известного шифрованного.

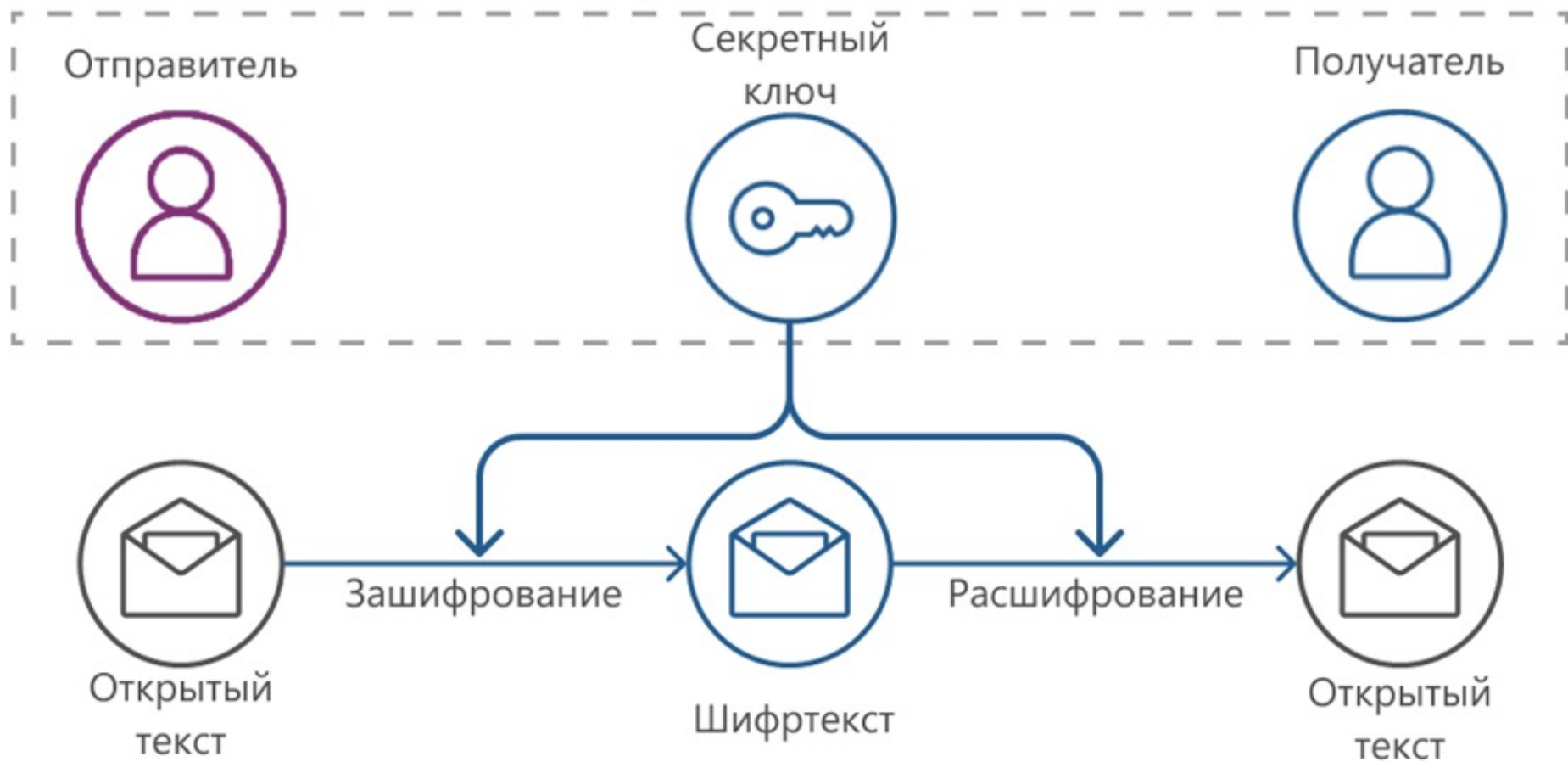
Термин «дешифрование» обычно применяют по отношению к процессу криптоанализа шифротекста (криптоанализ сам по себе, вообще говоря, может заключаться и в анализе шифросистемы, а не только зашифрованного ею открытого сообщения).

- **Криптографическая стойкость** — способность криптографического алгоритма противостоять криптоанализу.

Виды атак

- **По шифротексту.**
- **По известному тексту.** Это происходит, если Ева получает в свое распоряжение какие-либо открытые тексты, соответствующие ранее переданным зашифрованным. Сопоставляя пары «текст-шифротекст», Ева пытается узнать секретный ключ, чтобы с его помощью дешифровать все последующие сообщения от Алисы к Бобу.
- **По выбранному тексту.** Противник пользуется не только предоставленными ему парами «текст-шифротекст», но может и сам формировать нужные ему тексты и шифровать их с помощью того ключа, который он хочет узнать.

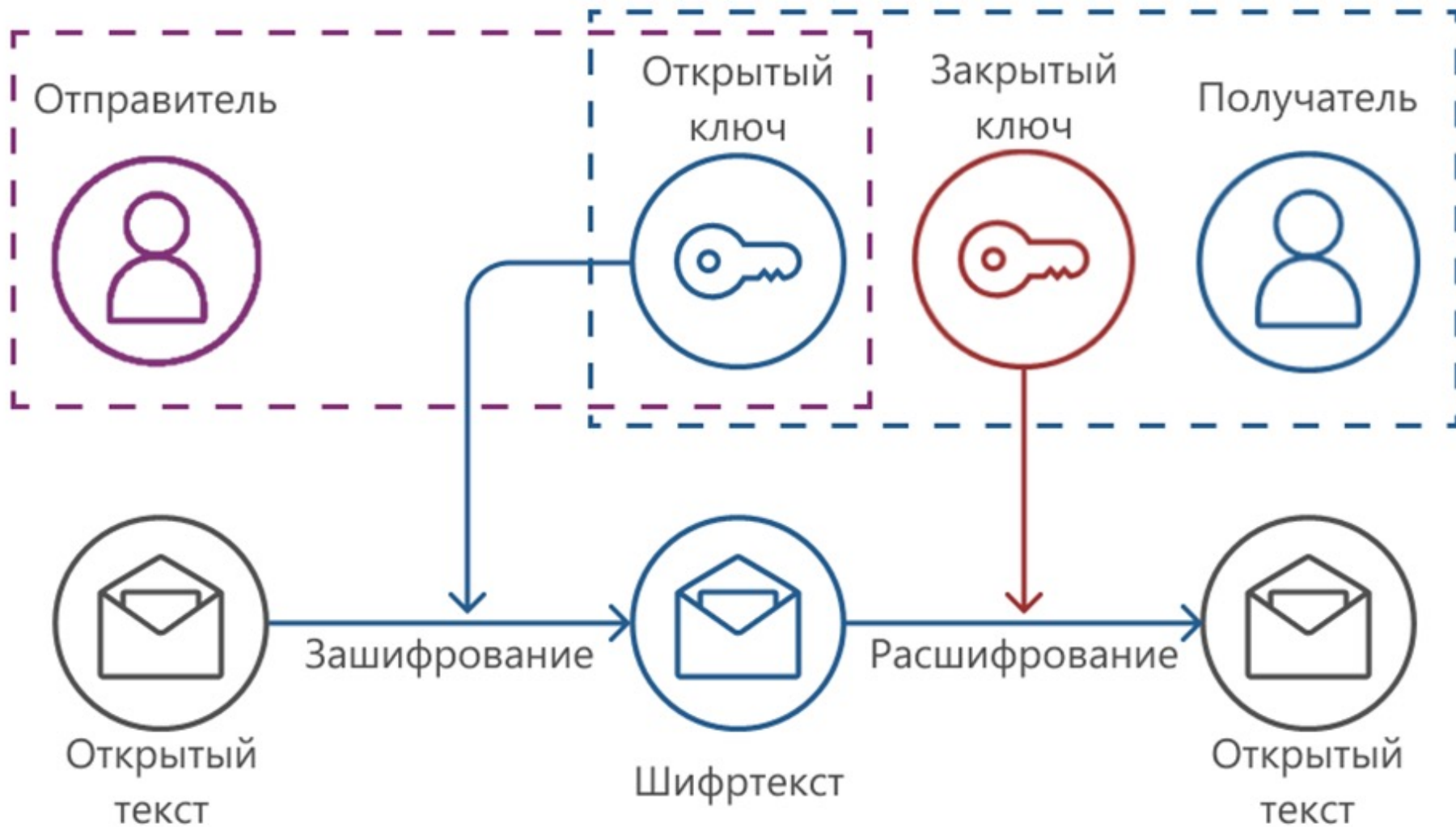
Симметричное шифрование



Примеры задач, решаемых криптографией с открытым ключом:

- Хранение паролей в компьютере.
- Верификация в системах ПВО (при пересечении самолетом границы радиолокатор спрашивает пароль).
- Удаленный доступ через компьютерные сети (например, при взаимодействии банка и клиента). Обычно в начале сеанса банк запрашивает у клиента имя, а затем секретный пароль, но злоумышленник может узнать пароль, так как линия связи открытая.

Асимметричное шифрование



Асимметричное шифрование в криптографии основано на **односторонних функциях** (one-way functions).

Зачастую используется функция возведения в степень по модулю:

$$y = a^x \bmod p$$

Обратная функция (дискретный логарифм) обозначается:

$$x = \log_a y \bmod p$$

При больших модулях (более 512 бит) такую обратную функцию (дискретный логарифм) практически невозможно вычислить.

Элементы теории чисел

Определение 2.2. Целое положительное число p называется *простым*, если оно не делится ни на какое другое число, кроме самого себя и единицы.

Пример 2.3. Числа 11, 23 — простые; числа 27, 33 — составные (27 делится на 3 и на 9, 33 делится на 3 и на 11).

Теорема 2.3 (основная теорема арифметики). Любое целое положительное число может быть представлено в виде произведения простых чисел, причем единственным образом.

Пример 2.4. $27 = 3 \cdot 3 \cdot 3$, $33 = 3 \cdot 11$.

Определение 2.3. Два числа называются *взаимно простыми*, если они не имеют ни одного общего делителя кроме единицы.

Пример 2.5. Числа 27 и 28 взаимно просты (у них нет общих делителей кроме единицы), числа 27 и 33 — нет (у них есть общий делитель 3).

Определение 2.4 (функция Эйлера). Пусть дано целое число $N \geq 1$. Значение *функции Эйлера* $\varphi(N)$ равно количеству чисел в ряду $1, 2, 3, \dots, N - 1$, взаимно простых с N .

Пример 2.6.

$$\varphi(10) = ?$$

1, ~~2~~, 3, ~~4~~, ~~5~~, ~~6~~, 7, ~~8~~, 9,

$$\varphi(10) = 4$$

$$\varphi(12) = ?$$

1, ~~2~~, ~~3~~, ~~4~~, 5, ~~6~~, 7, ~~8~~, ~~9~~, ~~10~~, 11

$$\varphi(12) = 4$$

(здесь зачеркнуты числа, не взаимнопростые с аргументом).

Утверждение 2.4. Если p — простое число, то $\varphi(p) = p - 1$.

Доказательство. В ряду $1, 2, 3, \dots, p-1$ все числа взаимно просты с p , так как p — простое число и по определению не делится ни на какое другое число.

Утверждение 2.5. Пусть p и q — два различных простых числа ($p \neq q$). Тогда $\varphi(pq) = (p-1)(q-1)$.

Доказательство. В ряду $1, 2, \dots, pq-1$ не взаимнопростыми с pq будут числа

$$p, 2p, 3p, \dots, (q-1)p$$

и

$$q, 2q, 3q, \dots, (p-1)q.$$

Всего таких чисел будет $(q-1) + (p-1)$. Следовательно, количество чисел, взаимнопростых с pq , будет $pq-1 - (p-1) - (q-1) = pq - q - p + 1 = (p-1)(q-1)$.

Теорема 2.6 (Ферма). Пусть p — простое число и $0 < a < p$.
Тогда

$$a^{p-1} \bmod p = 1.$$

Пример 2.7. $p = 13, a = 2$;

$$2^{12} \bmod 13 = (2^2)^2 \cdot \left((2^2)^2\right)^2 \bmod 13 = 3 \cdot 9 \bmod 13 = 1,$$

$$10^{10} \bmod 11 = 10^2 \cdot \left((10^2)^2\right)^2 \bmod 11 = 1 \cdot 1 = 1.$$

Теорема 2.7 (Эйлер). Пусть a и b — взаимно простые числа. Тогда

$$a^{\varphi(b)} \bmod b = 1.$$

Теорема Ферма является частным случаем теоремы Эйлера, когда b — простое число.

Пример 2.8.

$$\varphi(12) = 4,$$

$$5^4 \bmod 12 = (5^2)^2 \bmod 12 = (1^2)^2 \bmod 12 = 1.$$

$$\varphi(21) = 2 \cdot 6 = 12,$$

$$2^{12} \bmod 21 = 2^4 \cdot (2^4)^2 \bmod 21 = 16 \cdot 4 \bmod 21 = 1.$$

Нам понадобится еще одна теорема, близкая к теореме Эйлера.

Теорема 2.8. *Если p и q — простые числа, $p \neq q$ и k — произвольное целое число, то*

$$a^{k\varphi(pq)+1} \bmod (pq) = a. \quad (2.12)$$

Пример 2.9. Возьмем $p = 5$, $q = 7$. Тогда $pq = 35$, а функция Эйлера — $\varphi(35) = 4 \cdot 6 = 24$. Рассмотрим случай $k = 2$, т.е. будем возводить числа в степень $2 \cdot 24 + 1 = 49$. Получим

$$9^{49} \bmod 35 = 9, \quad 23^{49} \bmod 35 = 23.$$

Это не удивительно, так как каждое из чисел 9 и 23 взаимно просто с модулем 35, и по теореме Эйлера $9^{24} \bmod 35 = 1$, $23^{24} \bmod 35 = 1$. Однако теорема 2.8 остается верной и для следующих чисел:

$$10^{49} \bmod 35 = 10, \quad 28^{49} \bmod 35 = 28,$$

в то время как теорема Эйлера для них не применима (каждое из чисел 10 и 28 не взаимно просто с модулем 35 и $10^{24} \bmod 35 = 15$, $28^{24} \bmod 35 = 21$). $\square$

Алгоритм Евклида

Для нахождения наибольшего общего делителя можно использовать следующий алгоритм, известный как алгоритм Евклида.

Алгоритм 2.1. АЛГОРИТМ ЕВКЛИДА

ВХОД: Положительные целые числа a, b , $a \geq b$.

ВЫХОД: Наибольший общий делитель $\gcd(a, b)$.

1. WHILE $b \neq 0$ DO
2. $r \leftarrow a \bmod b$, $a \leftarrow b$, $b \leftarrow r$.
3. RETURN a .

Пример 2.11. Покажем, как с помощью алгоритма Евклида вычисляется $\gcd(28, 8)$:

$a :$	28	8	4
$b :$	8	4	0
$r :$	4	0	

Обобщенный алгоритм Евклида

Теорема 2.9. Пусть a и b — два целых положительных числа. Тогда существуют целые (не обязательно положительные) числа x и y , такие, что

$$ax + by = \gcd(a, b). \quad (2.13)$$

Обобщенный алгоритм Евклида служит для отыскания $\gcd(a, b)$ и x, y , удовлетворяющих (2.13). Введем три строки $U = (u_1, u_2, u_3)$, $V = (v_1, v_2, v_3)$ и $T = (t_1, t_2, t_3)$. Тогда алгоритм записывается следующим образом.

Алгоритм 2.2. ОБОБЩЕННЫЙ АЛГОРИТМ ЕВКЛИДА

ВХОД: Положительные целые числа a, b , $a \geq b$.

ВЫХОД: $\gcd(a, b)$, x , y , удовлетворяющие (2.13).

1. $U \leftarrow (a, 1, 0)$, $V \leftarrow (b, 0, 1)$.
2. WHILE $v_1 \neq 0$ DO
3. $q \leftarrow u_1 \operatorname{div} v_1$;
4. $T \leftarrow (u_1 \bmod v_1, u_2 - qv_2, u_3 - qv_3)$;
5. $U \leftarrow V$, $V \leftarrow T$.
6. RETURN $U = (\gcd(a, b), x, y)$.

Результат содержится в строке U .

Операция div в алгоритме — это целочисленное деление

$$a \operatorname{div} b = \lfloor a/b \rfloor.$$

Пример 2.12. Пусть $a=28$, $b=19$. Найдем числа x и y , удовлетворяющие (2.13).

U				28	1	0	
V	U			19	0	1	
T	V	U		9	1	-1	$q = 1$
	T	V	U	1	-2	3	$q = 2$
		T	V	0	19	-28	$q = 9$

Инверсия

Рассмотрим одно важное применение обобщенного алгоритма Евклида. Во многих задачах криптографии для заданных чисел s , t требуется находить такое число $d < t$, что

$$cd \bmod t = 1. \quad (2.14)$$

Отметим, что такое d существует тогда и только тогда, когда числа s и t взаимно простые.

Определение 2.6. Число d , удовлетворяющее (2.14), называется *инверсией с по модулю m* и часто обозначается $c^{-1} \bmod m$.

Данное обозначение для инверсии довольно естественно, так как мы можем теперь переписать (2.14) в виде

$$cc^{-1} \bmod m = 1.$$

Умножение на c^{-1} соответствует делению на c при вычислениях по модулю m . По аналогии можно ввести произвольные отрицательные степени при вычислениях по модулю m :

$$c^{-e} = (c^e)^{-1} = (c^{-1})^e \pmod{m}.$$

Пример 2.13. $3 \cdot 4 \bmod 11 = 1$, поэтому число 4 — это инверсия числа 3 по модулю 11. Можно записать $3^{-1} \bmod 11 = 4$. Число $5^{-2} \bmod 11$ может быть найдено двумя способами:

$$5^{-2} \bmod 11 = (5^2 \bmod 11)^{-1} \bmod 11 = 3^{-1} \bmod 11 = 4,$$

$$5^{-2} \bmod 11 = (5^{-1} \bmod 11)^2 \bmod 11 = 9^2 \bmod 11 = 4.$$

При вычислениях по второму способу мы использовали равенство $5^{-1} \bmod 11 = 9$. Действительно, $5 \cdot 9 \bmod 11 = 45 \bmod 11 = 1$.

Покажем, как можно вычислить инверсию с помощью обобщенного алгоритма Евклида. Равенство (2.14) означает, что для некоторого целого k

$$cd - km = 1. \quad (2.15)$$

Учитывая, что c и m взаимно просты, перепишем (2.15) в виде

$$m(-k) + cd = \gcd(m, c), \quad (2.16)$$

Пример 2.14. Вычислим $7^{-1} \bmod 11$. Используем такую же схему записи вычислений, как в примере 2.12 :

$$\begin{array}{rcl}
 11 & 0 & \\
 7 & 1 & \\
 4 & -1 & q = 1 \\
 3 & 2 & q = 1 \\
 1 & -3 & q = 1 \\
 0 & 11 & q = 3.
 \end{array}$$

Получаем $d = -3$ и $d \bmod 11 = 11 - 3 = 8$, т.е. $7^{-1} \bmod 11 = 8$.
 Проверим результат: $7 \cdot 8 \bmod 11 = 56 \bmod 11 = 1$.

Проверка числа на простоту за $O(\sqrt{P})$

Чтобы определить, является ли данное число P простым, безусловно, достаточно написать простой цикл поиска делителей числа P :

```
bool prime (long long p){  
    for (long long i = 2; i <= sqrt(p); i++)  
        if (p%i == 0)  
            return false;  
    return true;  
}
```

Данная функция проверки числа на простоту достаточно эффективна. Однако иногда нужно уметь проверять число на простоту быстрее.

Проверка числа на простоту за $O(K \cdot \log P)$

Чтобы проверить число **P** на простоту с достаточно хорошей вероятностью безошибочности, достаточно **K** раз (например, 100 раз) проверить случайное число **a** тестом Ферма.

Теорема 2.6 (Ферма). Пусть p — простое число и $0 < a < p$.
Тогда

$$a^{p-1} \bmod p = 1.$$

1. Если для основания a и модуля n выполняется $a^{n-1} \equiv 1 \pmod{n}$, то n может быть простым числом.
2. Если $a^{n-1} \not\equiv 1 \pmod{n}$, то n — однозначно составное.

При проверке числа на простоту тестом Ферма выбирают несколько чисел **a**. Чем больше количество **a**, для которых $a^{n-1} \bmod p = 1$, тем больше шансы, что число **p** - простое.

Однако существуют составные числа, для которых сравнение $a^{n-1} \bmod p = 1$ выполняется для всех **a**, взаимно простых с **p**.

Это числа Кармайкла, их бесконечное множество, наименьшее число Кармайкла — 561.

Число Кармайкла - это составное число **p**, такое что $a^{n-1} \bmod p = 1$ для всех целых чисел **a**, взаимно простых с **p**.

Примеры чисел Кармайкла:

**561, 1105, 1729, 2465, 2821, 6601, 8911, 10585, 15841, 29341,
41041, 46657, 52633, 62745, 63973, 75361, ...**

**Тем не менее, тест Ферма довольно эффективен для
обнаружения составных чисел.**

Тест Ферма эффективен для обнаружения составных чисел при верном выборе параметра **K** .

Данный параметр означает количество различных чисел **a** из диапазона **$0 < a < p$** , которые необходимо перебрать для проверки числа **p** на простоту.

$$P(\text{число простое}) = 1 - \frac{1}{2^K},$$

где **K** — количество чисел **a** (кол — во итераций проверки).

$$\text{При } K = 1: P(\text{число простое}) = 1 - \frac{1}{2^1} = \frac{1}{2}$$

$$\text{При } K = 2: P(\text{число простое}) = 1 - \frac{1}{2^2} = \frac{3}{4}$$

$$\text{При } K = 10: P(\text{число простое}) = 1 - \frac{1}{2^{10}} = \frac{1023}{1024} \approx 1$$

Теорема 2.6 (Ферма). Пусть p — простое число и $0 < a < p$.
Тогда

$$a^{p-1} \bmod p = 1.$$

```
bool ferma (long long p){  
    if (p == 2)  
        return true;  
    srand (time (NULL));  
    for (int i = 0; i < 100; i++){  
        long long a = (rand() % (p - 2)) + 2;  
        if (gcd (a, p) != 1)  
            return false;  
  
        if (pows (a, p-1, p) != 1)  
            return false;  
    }  
    return true;  
}
```

Числа **a** и **p** должны быть взаимно просты. Если это условие не выполняется, то число **p** — заведомо непростое.

Данная функция проверки использует функции нахождения НОД, а также быстрого возведения в степень по модулю.

Тест Ферма достаточно эффективен для обнаружения составных чисел при верном выборе параметра ***K***.

Однако числа Кармайкла успешно проходят тест Ферма, хотя не являются простыми.

Отчасти их можно отсеять за счёт проверки наибольшего общего делителя, но такая проверка не имеет хорошего теоретического обоснования вероятности отсеечения, поэтому необходимо рассмотреть и другие тесты.

Методы взлома шифров, основанных на дискретном логарифмировании

Вычисления параметров во многих криптографических шифрах основываются на односторонних функциях.

$$y = a^x \bmod p$$

Обратная функция (дискретный логарифм) обозначается:

$$x = \log_a y \bmod p$$

Можно вычислить **y** , если даны **a** и **x** , затратив не более, чем **$2 \log x$** операций.

Однако отыскание **x** по известным **a** и **y** , т.е. вычисление дискретного логарифма, — задача намного более трудная.

Достаточно рассматривать только показатели x , удовлетворяющие неравенству $0 \leq x \leq p - 1$.

Обозначим через t_y число операций умножения, необходимых для вычисления y в (3.1) по a и x , и будем для краткости называть t_y временем вычисления. Время возведения в степень по алгоритмам из разд. 2.1 не больше $2 \log x$, причем $x < p$. Отсюда

$$t_y \leq 2 \log p \quad (3.2)$$

при любом показателе степени x .

Задача отыскания x в (3.1) по данным a и y .

Оценим сложность прямого перебора:

Сначала вычислить a^1 и проверить, верно ли равенство $a^1 = y$. Если нет, то проверяем $a^2 = y$, если нет, то $a^3 = y$ и т.д. до a^{p-1} .

В среднем потребуется $(p - 1) / 2$ раз умножать на a и проверять равенство. Таким образом, время прямого перебора:

$$t_{\text{п.п.}} \approx p/2.$$

В методе «шаг младенца, шаг великана» время отыскания **х** существенно меньше:

$$t_{\text{ш.м.ш.в.}} \approx 2 \cdot \sqrt{p},$$

а в методе исчисления порядка это время еще меньше:

$$t_{\text{и.п.}} \approx c_1 \cdot 2^{c_2 \sqrt{\log p \log \log p}},$$

где c_1 , c_2 — некоторые положительные константы.

Чтобы сделать сравнение более наглядным, выразим время вычисления через длину числа p в (3.1). Обозначим эту длину в битах через n . При вычислениях по модулю p имеем $n \approx \log p$. Поэтому порядок трудоемкости (в смысле количества операций) упомянутых алгоритмов будет следующий:

$$\begin{aligned} t_y &\approx n, \\ t_{\text{п.п.}} &\approx 2^{n-1}, \\ t_{\text{ш.м.ш.в.}} &\approx 2^{n/2}, \\ t_{\text{и.п.}} &\approx 2^{c_2 \sqrt{n \log n}}, \end{aligned}$$

где $\approx$ означает «пропорционально».

Метод «шаг младенца, шаг великана»

В открытой литературе этот метод был впервые описан Дэниелом Шенксом (Daniel Shanks) в 1973 году.

Это один из первых методов, который показал, что задача вычисления дискретного логарифма может быть решена значительно быстрее, чем методом перебора.

Описание метода «шаг младенца, шаг великана»:

Шаг 1. Сначала берем два целых числа m и k , такие, что

$$mk > p. \quad (3.3)$$

Шаг 2. Вычислим два ряда чисел

$$y, \quad ay, \quad a^2y, \quad \dots, \quad a^{m-1}y \pmod{p}; \quad (3.4)$$

$$a^m, \quad a^{2m}, \quad \dots, \quad a^{km} \pmod{p} \quad (3.5)$$

(все вычисления проводятся по модулю p).

Шаг 3. Найдем такие i и j , для которых выполняется равенство

$$a^{im} = a^j y. \quad (3.6)$$

Утверждение 3.1. *Число*

$$x = im - j \quad (3.7)$$

является решением уравнения (3.1). Кроме того, целые числа i, j , удовлетворяющие (3.6), существуют.

Доказательство. Справедливость (3.7) следует из приводимой ниже символической цепочки равенств, где все вычисления даны по модулю p , а деление соответствует умножению на обратный элемент:

$$a^x = a^{im-j} = \frac{a^{im}}{a^j} = \frac{a^{im}y}{a^j y} = \frac{a^{im}y}{a^{im}} = y.$$

Докажем теперь, что числа i и j , удовлетворяющие (3.6), существуют. Для этого сведем все числа вида (3.7) в таблицу 3.1.

Т а б л и ц а 3.1. Распределение чисел вида $it - j$

$i \downarrow \quad j \rightarrow$	0	1	2	...	$t - 1$
1	t	$t - 1$	$t - 2$	...	1
2	$2t$	$2t - 1$	$2t - 2$	...	$t + 1$
...	...	...	...	...	...
k	kt	$kt - 1$	$kt - 2$	...	$(k - 1)t + 1$

Мы видим, что в таблице содержатся все числа от 1 до kt . Значит, из (3.3) следует, что в таблице содержатся все числа от 1 до p . Таким образом, любой показатель степени $x < p$ будет содержаться в таблице, т.е. число x , удовлетворяющее (3.1), может быть представлено в виде (3.7) и всегда найдется в таблице, поэтому уравнение (3.6) всегда имеет решение.

Пример 3.1. Найдем решение уравнения $2^x \bmod 23 = 9$, используя метод «шаг младенца, шаг великана».

Выберем m, k . Пусть $m = 6, k = 4$. Мы видим, что (3.3) выполняется. Вычислим числа (3.4), (3.5):

$$(3.4) : \quad 9, 18, 13, 3, 6, 12;$$

$$(3.5) : \quad 18.$$

Дальнейшие вычисления не проводим, так как уже нашлись одинаковые числа в (3.4) и (3.5) при $i = 1, j = 1$. По (3.7) получаем

$$x = 1 \cdot 6 - 1 = 5.$$

Проверим: $2^5 \bmod 23 = 9$. Действительно, $x = 5$ есть решение.

Имея два описанных ранее ряда, можно быстро найти в них равные элементы, воспользовавшись одним из описанных ниже методов.

Метод с использованием сортировки:

Пусть есть два массива $A[m], B[k]$ состоящие соответственно из m и k элементов. Ниже представлен код на языке C++, который позволяет находить в этих массивах равные элементы с трудоемкостью $O(m \log_2 m + k \log_2 k)$.

```
int A[m], B[k];
sort(A, A+m);
sort(B, B+k);
int i=0, j=0;
while(i<m && j<k)
{
    if(A[i]<B[j])
        ++i;
    else if(A[i]>B[j])
        ++j;
    else break;
}
if(i<m && j<k)
    cout << i << " " << j;
```


Метод с использованием структур данных:

Пусть есть два массива $A[m]$, $B[k]$ состоящие соответственно из m и k элементов. Ниже представлен код на языке C++, который позволяет находить равные элементы с трудоемкостью $O(m \log_2 m + k \log_2 k)$. Либо, в зависимости от используемой структуры данных, с трудоемкостью $O(mC + kC)$, где C - некоторая константа.

```
int A[m], B[k];
map<int,int> dict;
for(int i=0; i<m; ++i)
    dict[A[i]]=i;
for(int i=0; i<k; ++i)
    if(dict.count(B[i]))
    {
        cout << i << " " << dict[B[i]];
        break;
    }
```

Утверждение 3.2. *Время вычислений по данному методу при больших p удовлетворяет неравенству*

$$t_{ш.м.ш.в.} \leq \text{const} \cdot \sqrt{p} \log^2 p. \quad (3.8)$$

(Здесь речь идет о полном времени вычислений, а не о числе умножений).

Доказательство. Мы можем взять

$$k = m = \lfloor \sqrt{p} \rfloor + 1, \quad (3.9)$$

так что, очевидно, (3.3) выполняется. Тогда в (3.4) и (3.5) потребуется не более $2\sqrt{p}$ операций умножения. Мы знаем, что для «обычных» («школьных») методов умножения и деления время вычисления результата для двух r -значных чисел пропорционально r^2 . У нас все числа берутся из множества $\{1, \dots, p\}$, значит, $r \leq \log p$, и время вычисления пропорционально $\log^2 p$. Отсюда мы сразу получаем время, затраченное на вычисление рядов (3.4) и (3.5). Однако мы не

учли все этапы алгоритма. Мы не учли время, требуемое для нахождения равных чисел в этих рядах. При больших k и m это далеко не простая задача. Она может быть решена следующим образом: сначала каждому числу припишем его номер в ряду и еще один бит, в котором указана принадлежность к ряду (3.4) или (3.5), затем преобразуем обе последовательности в список и отсортируем (упорядочим по величине). Длина общего ряда равна $k + m \approx 2\sqrt{p}$. Для лучших методов сортировки требуется $S \log S$ операций сравнения, где S — число элементов в списке (см., например, [1]). В нашем случае $S = 2\sqrt{p}$ и, следовательно, требуется $2\sqrt{p} \log(2\sqrt{p}) \approx \sqrt{p} \log p$ операций сравнения над словами длины $\log p$ бит, т.е. всего требуется порядка $\sqrt{p} \log^2 p$ операций. После сортировки объединенного ряда его надо просмотреть и найти два равных числа из разных рядов (3.4), (3.5), используя битовый признак. Таким образом, суммируя время вычисления на всех этапах, получаем (3.8).